AI Code Review — Options Evaluation for SourceFuse

> **Decision brief.** Evaluates every AI pull-request review option realistically available to us — our in-house GitHub Action, Anthropic's five review offerings, and GitHub Copilot code review — and recommends which to run org-wide.
>
> **Prepared 2026-08-05; external sources and production telemetry both current to 2026-08-03.** Every external claim is sourced from the vendor's own primary documentation, with URLs in Appendix C and the fetch date on each. Claims that could not be verified from a primary source are labelled **[unverified]** in-line rather than asserted. Our own figures are traced to the file and line that produce them.

Executive summary

Recommendation: keep our AI PR Review Action as the org-wide default reviewer, and upgrade the model rather than the platform when a repo needs frontier-model quality.

There is no single competitor to compare against. Anthropic ships **five** distinct review products with different hosting, pricing and capabilities; GitHub ships a sixth (Copilot code review) plus a separate paid product (GitHub Code Quality) that exists because Copilot code review *cannot gate a merge*. None of the six covers the operational requirements we already run in production: **JIRA story context, Backstage per-finding tracking, merge gating, re-run noise control, and AI verification of human replies.**

Option	Weighted score /100	Cost per review	Fits our stack today
Our AI PR Review Action	**88**	**$0 marginal** on the existing GLM Coding Pro plan (~$80/mo flat)	✅
Managed "Code Review" (Anthropic SaaS)	64	$15–25	❌ Team/Enterprise, Anthropic-billed, research preview, no ZDR
`code-review` plugin via claude-code-action	55	your API spend (~$15–25 observed)	⚠️ partial
`anthropics/claude-code-action` (DIY prompt)	54	your API spend	⚠️ partial
GitHub Copilot code review	53	**undisclosed** — seat + AI credits + Actions minutes	⚠️ partial

The three findings that matter most to a decision:

1. **No AI reviewer on the market can block a merge on its own.** Anthropic's action *"cannot approve pull requests"*; its managed service always returns a *"neutral conclusion so it never blocks merging"*; Copilot *"always leaves a 'Comment' review"*; and GitHub's own Code Quality product states that **AI-powered findings never block a PR** — only deterministic CodeQL rules do. Our action's `fail_on_critical` check-run gate is, as of today, the only merge enforcement available to us from any of these options.
2. **Only our action controls re-run noise.** Copilot's docs state plainly that on re-review *"Copilot may repeat the same comments again, even if they have been dismissed with the 'Resolve conversation' button or downvoted."* The `code-review` plugin's entire re-run strategy is to abort if Claude has already commented. Anthropic's managed service handles it, but only via prompt tuning you write yourself in `REVIEW.md`.
3. **Cost is the widest gap and it runs in our favour — at measured volume.** Production telemetry across 6 repositories over 18 weeks shows **~2,150 workflow runs per month, ~1,500 of them completed reviews** (§4.2). We pay a flat **~$80/month** for that, on self-hosted runners that bill no GitHub Actions minutes. The same work through Anthropic's managed service at $15–25 per review would cost **$270,000–450,000 per year.** Copilot's per-review cost is not published at all — GitHub does not disclose the model, the credits, or the Actions minutes consumed, so cost per PR is not forecastable from public data.

This is a production system, not a pilot. 2,768 runs since April, ~499 per week sustained since mid-July, peak 196 runs in a single day. The reliability picture is honest rather than flattering: 60.8% of runs complete successfully, 26.6% are cancelled by superseding pushes, 6.9% fail, and median review latency is 9.4 minutes. §6 and §7 name the three fixes that address it.

Honest counterweight: Anthropic and GitHub staff their products; we maintain ours. Three competitors ship a **false-positive verification pass** we do not have. Copilot ships **suggestion-acceptance-rate metrics** that are genuinely better than our tracking. §7 covers how to close both gaps without changing platform.

---

1. The landscape — seven offerings, three vendors

The question "why not just use Claude/Copilot for code review?" contains a hidden assumption: that there is one such product. There are seven, and they are not substitutes for each other.

#	Offering	What it actually is	Where it runs	Status (2026-08-03)
1	**Our AI PR Review Action**	Purpose-built review pipeline: 11 declared phases, 8 dimensions, full comment lifecycle, metrics, gating	our runners, our AI endpoint	in production across SourceFuse repos
2	**`anthropics/claude-code-action`**	General GitHub automation platform. Code review is a *recipe you write yourself* in the `prompt` input	your runners	**GA — v1.0 since 2025-08-26**, currently v1.0.183
3	**`code-review` plugin** (`code-review@claude-code-plugins`)	A 9-step multi-agent review command, self-hosted inside offering #2	your runners	public
4	**Managed "Code Review"** (code.claude.com)	Anthropic-hosted multi-agent reviewer with a verification pass	**Anthropic's infrastructure**	research preview; Team/Enterprise only; **not available with Zero Data Retention**
5	**`/code-review` (local)**	Free bundled Claude Code skill; `--comment` posts inline comments to a PR	a developer's machine	GA, any plan
6	**`/code-review ultra` (ultrareview)**	Cloud multi-agent review with independent verification	Anthropic sandbox	research preview
7	**GitHub Copilot code review**	GitHub's reviewer, driven by repository rulesets	**GitHub Actions runners** (ephemeral)	**GA** (Medium effort level still preview)

Adjacent, and important to the argument: GitHub Code Quality — GA since 2026-07-20, $10/active committer/month plus usage. It is the only GitHub product that can genuinely block a merge on quality grounds, and it does so *only* on deterministic CodeQL findings. Profile in §3.7.

1.1 What "reinventing the wheel" would actually mean

Offering #2's review behaviour is entirely prompt-driven. Its `docs/solutions.md` "Automatic PR Code Review" recipe — the closest thing to an off-the-shelf reviewer in the OSS action — is this, in full:

```
Please review this pull request with a focus on:
- Code quality and best practices
- Potential bugs or issues
- Security implications
- Performance considerations

Note: The PR branch is already checked out in the current working directory.

Use `gh pr comment` for top-level feedback.
Use `mcp__github_inline_comment__create_inline_comment` (with `confirmed: true`) to highlight specific code issues.
Only post GitHub comments - don't submit review text as messages.
```

That recipe is real, it runs unattended on `pull_request: [opened, synchronize]`, and it **can** be pointed at our GLM plan — see §3.2.1, which documents exactly how, and the trap that makes the obvious configuration fail silently.

So the starting wheel is free and, with effort, provider-portable. What it does **not** contain — and what you would be building and owning yourself, per repo, in prompt text with no guarantees — is the entire review-workflow layer: thread lifecycle, deduplication, severity taxonomy, re-run focus, reply verification, metrics, gating, JIRA. That layer is the product.

---

2. Head-to-head matrix

Legend: ✅ built-in · 🟡 partial / DIY / conditional · ❌ not available. Columns: **Ours** · **OSS** = `claude-code-action` DIY prompt · **Plugin** = `code-review` plugin · **Managed** = Anthropic SaaS · **Copilot** = GitHub Copilot code review. Offerings #5 and #6 (local `/code-review`, ultrareview) are developer-workstation tools rather than unattended CI reviewers and are profiled separately in §3.5.

2.1 Review engine

Capability	Ours	OSS	Plugin	Managed	Copilot
Purpose-built for code review	✅	❌ general automation	✅	✅	✅
Review dimensions	✅ 8 fixed categories, combined or 8 parallel specialists	🟡 whatever the prompt says	🟡 narrow by design: compile/parse errors, definite logic bugs, CLAUDE.md violations	✅ multi-agent, correctness-focused	🟡 undocumented mix; *"reviews code written in any language"*
Severity taxonomy	✅ 5 levels (critical/high/medium/low/nit)	🟡 prompt-defined	❌ none — binary validated/discarded	✅ 3 levels (🔴 Important / 🟡 Nit / 🟣 Pre-existing)	❌ none documented; formatting instructions explicitly unsupported
Multi-agent pipeline	✅ 8 specialists (separate mode)	❌	✅ 4 parallel + N validators	✅	🟡 agentic tool-calling, single reviewer
False-positive verification pass	❌ *(roadmap — see §7)*	🟡 Haiku classifies buffered inline comments as real-vs-test	✅ per-issue validation subagents	✅ *"checks candidates against actual code behavior"*	❌ *"has a risk of hallucination"*
Deduplication	✅ programmatic (Levenshtein/Jaccard) + AI semantic merge in separate mode	❌	🟡 within a run only	✅ built-in	❌
Review profiles / intensity	✅ strict / standard / minimal + 8 per-agent toggles	❌	❌	❌ (REVIEW.md tuning only)	🟡 Low / Medium effort (Medium in preview)
Custom review rules	✅ `system_prompt_append`, framework appends, full override	✅ prompt / CLAUDE.md / `--system-prompt`	🟡 CLAUDE.md only	✅ REVIEW.md (highest priority) + CLAUDE.md	✅ `copilot-instructions.md`, path-specific, AGENTS/CLAUDE/REVIEW.md, skills, MCP
Framework-aware review (Angular, LoopBack4)	✅ auto-detected prompt additions + DI-aware context	❌	❌	❌	❌
Scope of findings	added (+) diff lines only	prompt-defined	the diff	diff + flags pre-existing separately	the diff

2.2 Codebase context

Capability	Ours	OSS	Plugin	Managed	Copilot
Sees beyond the diff	✅ compiler-exact import graph: tsconfig aliases, npm workspaces, barrel re-exports, reverse dependencies, Angular siblings, LB4 DI bindings	✅ full checkout, agent reads what it wants	🟡 diff-focused by instruction	✅ *"full codebase context"*	✅ agentic `grep` / `rg` / `glob` / `view` over the repo
Cost-bounded context gathering	✅ hard caps in code (12 tool calls/run, 24 related files, 100k chars)	❌ capped only by `--max-turns` (default 10)	🟡 *"Only call a tool if it is required"*	n/a (flat fee absorbs it)	❌ (consumes credits + Actions minutes)
Hard file blind spots	❌ (defaults are configurable)	❌	❌	🟡 REVIEW.md skip rules	❌ **~45 named files + 16 globs permanently excluded** — incl. `tsconfig.json`, `*.d.ts`, `build.gradle`, `requirements.txt`, `vendor/**`, `generated/**`
JIRA story context (acceptance criteria in the review)	✅	❌	❌	❌	🟡 read-only via MCP, heuristically triggered
Reads instructions from the PR head branch	✅	✅	✅	✅	✅ (explicitly documented)

2.3 PR interaction & noise control

Capability	Ours	OSS	Plugin	Managed	Copilot
Inline comments on diff lines	✅	✅ (MCP tool, must be allow-listed + prompt-instructed)	✅	✅	✅
Committable code suggestions	✅ `suggestion` blocks, validated for no-ops/mismatch	🟡 prompt-dependent	✅ small fixes only, by rule	❌ ("Fix this" links instead)	✅
One summary comment, updated in place	✅ + old runs minimized, never deleted	🟡 `use_sticky_comment`	🟡 only when zero issues found	✅ review body + check run + annotations	✅ PR overview comment
Re-run awareness (new inline comments narrowed after the first review)	✅ "Re-run #N", critical/high + docs only	❌ stateless per run	❌ **reviews a PR exactly once** — verified: a second run on a changed diff posted nothing	🟡 tunable via REVIEW.md "convergence"	❌
Never re-posts a comment the developer already resolved	✅	🟡 stateless — likely to repeat	n/a (aborts instead)	✅	❌ **documented behaviour** — *"may repeat the same comments again, even if they have been dismissed … or downvoted"*
Auto-resolve threads when fixed	✅	❌	❌	✅ (push-triggered mode)	❌
Reopen resolved threads on regression (critical/high, never over a human justification)	✅	❌	❌	❌	❌
AI verification of human replies ("this is by design" → checked against code, answered, resolved if valid)	✅	❌	❌	❌ — *"Replying to an inline comment does not prompt Claude to respond"*	❌ — *"they won't be visible to Copilot, and Copilot won't reply"*
AI PR description + validated Mermaid diagrams	✅	❌	❌	❌	🟡 PR summaries — **Copilot Enterprise only**
Recurring bot-comment cleanup (SonarQube spam etc.)	✅	❌	❌	❌	❌
"Fix this" links / hand-off to a coding agent	❌	✅ `include_fix_links`	❌	✅	✅ "Fix with Copilot" → cloud agent
Interactive @mention Q&A on the PR	❌ (review-only by design)	✅	❌	🟡 `@claude review` trigger only	✅ `@copilot` (cloud agent, separate product)
👍/👎 feedback loop	❌	❌	❌	✅ tunes the reviewer post-merge	🟡 collected, but a 👎 comment can still recur

2.4 Governance, metrics & gating

Capability	Ours	OSS	Plugin	Managed	Copilot
Structured outputs for workflow steps	✅ **23** named outputs (counts, tokens, cost, status, thread activity)	🟡 one `structured_output` JSON, only with `--json-schema`	❌	🟡 severity counts parseable from the check run	❌
Central tracking: every run + every finding POSTed	✅ ~50 fields + all findings, incl. skipped/failed/cancelled runs	❌	❌	🟡 Anthropic-hosted dashboard	❌ no findings export API; standard PR-comment REST works
Aggregate analytics dashboard	🟡 Backstage (we build it)	❌	❌	✅ hosted (`claude.ai/analytics/code-review`)	✅ metrics API incl. **suggestion acceptance rate**
Token + estimated cost per run, in the PR comment	✅	❌	❌	🟡 in Anthropic's dashboard, not the PR	❌ per-review cost not disclosed at all
Merge gating	✅ opt-in (`fail_on_critical` + `fail_threshold` → failed check run)	🟡 buildable from structured output	❌	🟡 neutral by design, **but** a `bughunter-severity` JSON in the check run enables DIY gating	❌ needs **GitHub Code Quality** ($10/committer) — and even there, AI findings never gate
Formal PR approval / request-changes	❌	❌ *"Claude cannot approve pull requests"*	❌	❌ by design	❌ *"always leaves a 'Comment' review"*

> **Note on our own gating, stated plainly:** our inline comments are submitted through the Reviews API with `event: 'COMMENT'` (`src/github/inline-reviewer.ts:110,322`). We never send `APPROVE` or `REQUEST_CHANGES` either. Our enforcement is the **check run** — `fail_on_critical` calls `setFailed`, and branch protection must be configured to require that check. This is a real difference from the others (they have no gate at all), but it is a check-run gate, not a review verdict.

2.5 Providers, cost & operations

Capability	Ours	OSS	Plugin	Managed	Copilot
Works with our existing z.ai/GLM-5.2 keys	✅ first-class, documented, validated inputs	🟡 yes but **wholly undocumented and unsupported** — and z.ai's own documented variable (`ANTHROPIC_AUTH_TOKEN`) is silently dropped; see §3.2.1	🟡 same, plus Haiku/Sonnet/Opus tier remapping	❌ Anthropic-billed only	❌
Any OpenAI-compatible endpoint	✅ dedicated `openai` dialect	🟡 Anthropic-compatible base URLs only	🟡	❌	❌
Bedrock / Vertex / Foundry	❌	✅ (OIDC)	✅	❌ (billed by Anthropic regardless)	❌
Choose or even *know* the model	✅ any, per repo	✅	✅	❌	❌ *"Model switching is not supported"*; *"the model … is not disclosed"*
Keyless auth	❌ static token	✅ workload identity federation (GitHub OIDC)	✅	n/a	n/a
Model fallback chain	✅ comma-separated, auto-latching	❌ (`fallback_model` removed)	❌	n/a	n/a
Rate-limit resilience	✅ patient 429 budget (10s escalating ×1.5 to a 120s ceiling, up to 5h **per AI call**)	🟡 Claude Code defaults	🟡	❌ *"doesn't retry on its own"*	🟡 [unverified]
Declared fault-tolerance contract	✅ per phase: best-effort work never fails the run; failures still report metrics	🟡	🟡	🟡 error comment, neutral check	🟡 degrades to a *"more limited review"* if Actions is unavailable
Cancel gracefully when the PR closes mid-review	✅	❌	❌	n/a	n/a
Pre-flight AI-endpoint health probe	✅	❌	❌	n/a	n/a
Skip oversized PRs instead of timing out	✅ `max_files_to_review` (default 50)	❌	❌	🟡 spend-cap comment	🟡 [unverified] no published limit
Configurable file include/exclude filters	✅ 17 built-in defaults, user patterns append	🟡 via workflow `paths:` / prompt	❌	✅ REVIEW.md skip rules	🟡 content exclusions (Business/Enterprise), plus a fixed non-configurable exclusion list
Consumes billable GitHub Actions minutes	✅ **no** — we run on SourceFuse self-hosted runners	🟡 depends on your runners	🟡	✅ no (runs on Anthropic infra)	❌ **yes**, charged to the repository on private repos since 2026-06-01
Data residency	✅ our runners + our chosen endpoint	✅ your runners	✅	❌ Anthropic infra; **unavailable with ZDR**	🟡 US/EU on GHEC-with-data-residency, **+10% AI credit cost**
Latency	**median 9.4 min, 11 min for successful reviews, p95 37 min** — measured across 2,404 production runs	varies with the agent loop	varies	*"20 minutes on average"*	*"usually … less than 30 seconds"* (pre-dates the agentic architecture; no figure published for Medium effort)

2.6 Setup, triggers & ownership

Capability	Ours	OSS	Plugin	Managed	Copilot
One-time setup	1 secret + 1 workflow file (`contents: write` + `pull-requests: write`)	`/install-github-app`, or App install + key + workflow	same as OSS	repo toggle in Claude settings — no CI at all	**repository ruleset** — no CI file needed
Identity / permissions	`GITHUB_TOKEN`; `contents: write` needed for thread resolve	Claude GitHub App or custom App	same	Claude App (Contents/Issues/PRs read+write)	`copilot-pull-request-reviewer[bot]`
Trigger model	`pull_request` events (opened/synchronize/reopened) — unattended	`@claude` mention, any event, or cron	same	once per PR / every push / `@claude review [once|always]`	ruleset: on open; "Review new pushes" and "Review draft PRs" both opt-in
Runs unattended on every PR out of the box	✅	🟡 wire the workflow + prompt yourself	🟡	✅	✅
Maintenance owner	**SourceFuse (us)**	Anthropic (action) + you (prompt)	Anthropic (plugin) + you	Anthropic	GitHub

---

3. Deep profiles

3.1 Our AI PR Review Action

What it is. A Docker GitHub Action implementing a fixed review pipeline. Not an agent that might review — a pipeline that always does, with declared behaviour on every failure path.

Pipeline — 11 phases, each with a declared fail-safety contract (`src/pipeline/orchestrator.ts`, contract in `src/pipeline/phase.ts`):

Phase	Critical?	On failure
Startup (progress comment, minimize old summaries, bot cleanup)	best-effort	warn, continue
AI pre-flight probe (16 tokens, 45s, resolves + latches the working model)	gated	posts a failure comment naming the env vars, reports `ai_unreachable`, exits
Context gathering (diff, files, related context, JIRA, repo context)	**critical**	failure comment, Backstage `failed`, rethrow
Review agents (`Promise.allSettled` — individual agent failures tolerated)	**critical**	rethrow only if *every* agent failed
Consolidation (programmatic dedup → AI semantic merge in separate mode)	**critical**	AI failure falls back to programmatic dedup
Summary comment	**critical**	rethrow — "a review nobody can see is a failed review"
Reply handling · Inline comments · PR description · Tracking metrics · Backstage report · Job summary	best-effort	warn, continue

Two guards short-circuit cleanly rather than failing: `> max_files_to_review` (default 50) posts a friendly skip, and a zero-agent configuration skips. A PR-state watcher polls every 30s and exits *neutrally* if the PR closes or merges mid-review.

Review engine. 8 categories × 5 severities, all derived from one taxonomy file. Two modes: `combined` (one comprehensive agent, default) and `separate` (up to 8 parallel specialists, selected by profile `strict` / `standard` / `minimal` = 8/6/2 agents, individually overridable). Self-healing response handling: compact diff-only retry on context overflow, thinking-off + 2× output escalation on truncation, a JSON-repair turn, and per-file batching (up to 10 batches) when a PR cannot fit one prompt.

Context — the strongest technical differentiator. Primary engine shallow-fetches the PR head SHA into the container and uses the TypeScript compiler (ts-morph) for *exact* import resolution: monorepo-safe `tsconfig` paths incl. `extends`, npm workspace packages, barrels followed to the actual defining file, reverse dependencies (callers) confirmed by compiler-resolved imports, and declaration skeletons for large files. Candidates whose symbols appear in added diff lines are ranked up. Every included file carries a machine-generated reason (`imported`, `barrel-reexport`, `di-binding`, `template`, `caller`, …) rendered into the prompt. Framework heuristics resolve Angular `templateUrl` / `styleUrls` siblings and LoopBack4 string-key `@inject()` bindings. A GitHub-API static graph is the fallback engine if local acquisition fails. On top sits a **bounded** agentic tool loop (`read_file`, `grep`, `find_references`, `list_dir`) hard-capped at 2 rounds / 6 calls per review and **12 calls per run** — deliberately not an open agent loop.

Governance. 23 named action outputs; a Backstage POST carrying ~50 fields plus every finding, sent on **every** run including skipped, failed and cancelled ones; token/cost figures printed in the PR comment itself; optional merge gating.

Where it runs. A Docker action on **SourceFuse self-hosted runners**, calling our own AI endpoint. Two consequences: code and prompts never leave infrastructure we control, and the action consumes **no billable GitHub Actions minutes** — unlike Copilot code review, whose agentic capabilities run on Actions runners billed to the repository.

What it deliberately does not do. No code writing, no commits, no PR creation. No `@mention` interactivity — human replies are processed on the *next* run, not in real time. No formal approve/request-changes. No false-positive verification pass; our prompts explicitly bias toward flagging ("When in doubt, FLAG IT"). No feedback learning. TypeScript/JavaScript-first — the compiler engine, skeletons, barrels and framework depth are TS/JS-shaped; other languages get diff plus raw file text. No Bedrock/Vertex/Foundry. No hosted dashboard (Backstage is ours to build). Docker action ⇒ Linux runners only. State lives entirely in PR comment markers — delete the comments and it forgets.

3.2 `anthropics/claude-code-action` (OSS)

What it is. A general automation platform, GA at v1.0 since 2025-08-26 (now v1.0.183), with **39 inputs** and 5 outputs. Its own `action.yml` describes it as *"Flexible GitHub automation platform with Claude. Auto-detects mode based on event type."* Mode detection: a `prompt` ⇒ agent mode; no prompt but an `@claude` mention ⇒ tag mode; neither ⇒ nothing happens.

Review capability. Entirely what you write. `docs/solutions.md` ships 8 recipes, of which 4 are review-shaped (automatic PR review, path-filtered review, external-contributor review, security-focused review with its own prompt-defined CRITICAL/HIGH/MEDIUM/LOW scale). Inline comments come from one MCP tool, `mcp__github_inline_comment__create_inline_comment`, whose source comment states its design intent: *"Provides an inline comment tool without exposing full PR review capabilities, so that Claude can't accidentally approve a PR."*

The Haiku classifier — real, and narrower than it sounds. `classify_inline_comments` (default true) buffers any inline comment posted *without* `confirmed: true`, then after the session ends classifies each with `claude-haiku-4-5` as real review feedback vs a test/probe call, and posts only the real ones. It **fails open** on every error path (no API key, non-200, unparseable) — which means **Bedrock/Vertex users, who have no direct Anthropic key, get no filtering at all.**

Genuinely ahead of us: writes code and creates PRs, fixes bugs on request, interactive `@claude` Q&A, arbitrary cron/event automation, MCP servers, the skills/plugins ecosystem, Bedrock/Vertex/Foundry via OIDC, and **workload identity federation** — keyless GitHub-OIDC auth that removes the API-key secret entirely.

3.2.1 Can it run on our GLM-5.2 keys? Yes — but nothing about it is documented, and the obvious configuration fails silently

This matters because it is the strongest form of the "why not just use theirs?" question: if their action runs on our existing plan, provider flexibility is not our moat. Verified in detail:

Anthropic documents no third-party provider support at all. The GitHub Actions page documents exactly five authentication paths — Anthropic API key, OAuth token, workload identity federation, Amazon Bedrock, Google Cloud — and the repo's `docs/usage.md` makes **no mention** of `ANTHROPIC_BASE_URL`, `ANTHROPIC_AUTH_TOKEN`, custom endpoints, or non-Anthropic providers. There is no supported path, no compatibility statement, and no commitment not to break it.

It nonetheless works, through an undocumented passthrough. `action.yml` forwards a fixed list of ~75 environment variables into the Claude CLI subprocess, and `ANTHROPIC_BASE_URL`, `ANTHROPIC_CUSTOM_HEADERS` and `ANTHROPIC_DEFAULT_{SONNET,HAIKU,OPUS}_MODEL` are on it.

The trap. `ANTHROPIC_AUTH_TOKEN` — the variable z.ai's own Claude Code documentation tells you to set — is **not** on that list. Neither is `API_TIMEOUT_MS`, which z.ai also recommends. And the action's `env:` block **shadows** the calling workflow's job-level environment; its own source comment says so:

> *"these env vars are read directly from process.env by the Claude CLI subprocess. They must be listed explicitly here because this step's `env:` block shadows the calling workflow's job-level env vars (GitHub Actions composite action behavior)."*

So copying z.ai's documented setup into a workflow produces a job that looks correctly configured and silently authenticates as nobody.

The configuration that actually works (community-verified in `claude-code-action` discussion #673 and confirmed by the reporter) supplies the z.ai key through the `anthropic_api_key` **input** — which does reach the subprocess as `ANTHROPIC_API_KEY` — while the base URL goes through job-level `env:`:

```
jobs:
  review:
    runs-on: ubuntu-latest
    env:
      ANTHROPIC_BASE_URL: https://api.z.ai/api/anthropic   # passthrough works
    steps:
      - uses: actions/checkout@v6
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${{ secrets.ZAI_API_KEY }}     # NOT ANTHROPIC_AUTH_TOKEN
          claude_args: --model glm-5.2
```

Piloted, not theorised: what happened when we actually ran it

We ran this exact configuration on `sourcefuse/telescope-health-backend-api` (PR #4182), on our own self-hosted runners, against our GLM Coding Plan keys. **It took three attempts to get a review out of it**, and each failure is a finding.

Attempt 1 — blocked by a GitHub App requirement, before any AI call:

```
App token exchange failed: 401 Unauthorized - Claude Code is not installed on this repository. Please install the Claude Code GitHub App at https://github.com/apps/claude
```

With no `github_token` supplied, the action requests a GitHub OIDC token and exchanges it for a **Claude GitHub App** token. Out of the box, adopting it therefore requires an org admin to install Anthropic's GitHub App — independently of which model you point it at. Passing `github_token: ${{ secrets.GITHUB_TOKEN }}` skips the exchange.

Attempt 2 — ran to completion on GLM-5.2, and posted nothing. The run was reported green. Its result payload:

```
{"type":"result","subtype":"success","is_error":false,
 "duration_ms":1061515,        // 17.7 minutes
 "num_turns":38,
 "total_cost_usd":3.245332,
 "permission_denials_count":21}
```

The plugin declares its required tools in its own command frontmatter, but **that declaration is not honoured when the command is invoked through the action** — all 21 of its `gh` and inline-comment calls were denied, and the review was silently discarded. The job still reported success. A team running this unattended would see a green check and an empty review, with nothing in the PR to indicate the difference. The fix is an explicit `--allowedTools` in `claude_args` mirroring the plugin's frontmatter.

What attempt 2 does prove: the GLM wiring in this section is correct. The SDK options and init event both logged `"model": "glm-5.2"`, and the run consumed 38 turns against our z.ai endpoint. **Anthropic's official reviewer does run on our GLM plan.**

Attempt 3 — a real review, but a degraded one. Granting the plugin's own frontmatter tool list produced a genuine, good-quality review comment. It correctly verified the workflow's input validation, confirmed no script-injection vector, and caught a real `CLAUDE.md` violation (a missing mandatory JIRA ticket on the commits). But denials stayed at 21: `Task` was not in the list, so **the plugin's four parallel review agents and its per-issue validation subagents could never spawn.** What ran was a single-agent review wearing the pipeline's name. Granting `Task,Read,Glob,Grep,TodoWrite` dropped denials to 6.

Attempt 4 — the finding that matters most: it will not review the same PR twice. With the full toolset granted and a changed diff, the run completed successfully in 3.6 minutes, 20 turns — and **posted nothing at all**. This is consistent with the plugin's documented step-1 skip gate, which aborts the entire review when *"Claude has already commented on this PR."*

The operational consequence is severe and easy to miss: **the plugin reviews a pull request exactly once.** Push fixes in response to its feedback and the next run exits quietly with a green check. There is no re-review, no thread resolution, no convergence — the behaviour §3.3 describes, now confirmed on a live repository. For our push-heavy workflow, where 27% of runs are superseded by a further push (§4.2), a reviewer that only ever sees the first commit of a PR is not a viable primary.

Attempt	Tools granted	Result	Denials	Turns	Duration
1	—	failed: Claude GitHub App not installed	—	—	—
2	none	**green check, review silently discarded**	21	38	17.7 min
3	plugin frontmatter	review posted, but single-agent (no `Task`)	21	42	13.2 min
4	+ `Task,Read,Glob,Grep`	**nothing posted — already reviewed once**	6	20	3.6 min

Figures worth carrying into the cost discussion. 13–18 minutes for a single review of a 117-line diff, against a measured median of 9.4 minutes for our own action (§4.2) and Anthropic's 20-minute average for the managed service. The `total_cost_usd` the SDK reports ($0.72–3.25 per attempt) is computed at *Claude* list prices; on our flat GLM plan the real marginal cost is quota, not dollars — but 38–42 turns for one small diff shows how much more an unbounded agent loop consumes than our single-pass design.

One further catch for the `code-review` plugin specifically. Its pipeline requests Haiku, Sonnet and Opus subagents by tier name (§3.3). On a GLM endpoint those aliases must each be remapped with `ANTHROPIC_DEFAULT_HAIKU_MODEL`, `ANTHROPIC_DEFAULT_SONNET_MODEL` and `ANTHROPIC_DEFAULT_OPUS_MODEL`, or the tiering silently misroutes.

Bottom line for the decision. Provider portability is *achievable* on their action but is undocumented, unsupported, dependent on an env-var allow-list that Anthropic can change in any release, and requires knowing that the vendor's own documented variable is the wrong one. Ours takes `anthropic_auth_token` and `anthropic_base_url` as **first-class, validated, documented inputs**, adds a second `openai` wire dialect for endpoints that are not Anthropic-compatible, and supports a comma-separated model fallback chain. The capability gap is narrow; the **supportability** gap is not.

Verbatim from its own `docs/capabilities-and-limitations.md`, "What Claude Cannot Do": *"Submit PR Reviews: Claude cannot submit formal GitHub PR reviews"* · *"Approve PRs: For security reasons, Claude cannot approve pull requests"* · *"Post Multiple Comments: Claude only acts by updating its initial comment"* · no cross-repo access · no Bash without allow-listing · *"Cannot merge branches, rebase, or perform other git operations beyond pushing commits."*

Fairness caveat: that file is partially stale — the single-comment limitation is contradicted by the shipped inline-comment MCP tool. The rest is current and accurate.

3.3 The `code-review` plugin

What it is. One markdown command file (`plugins/code-review/commands/code-review.md`) in `anthropics/claude-code`, run inside the OSS action. It is a genuine 9-step multi-agent pipeline, and it is the most *quality-focused* of Anthropic's self-hosted options:

1. Haiku gate — skip if closed, draft, trivial, **or if Claude has already commented on this PR**
2. Haiku — locate relevant `CLAUDE.md` files
3. Sonnet — summarize the PR
4. **Four agents in parallel**: 2× Sonnet CLAUDE.md-compliance auditors, 2× Opus bug hunters
5. **Per-issue validation subagents** (Opus for bugs, Sonnet for CLAUDE.md) — the verification pass 6–9. Filter unvalidated issues → dry-run list → post

Deliberately narrow scope. Verbatim: *"**CRITICAL: We only want HIGH SIGNAL issues.**"* It flags only code that will fail to compile/parse, code that will definitely produce wrong results, and unambiguous CLAUDE.md violations you can quote. It explicitly does **not** flag code style, input-dependent issues, or subjective improvements — and has **no security, performance or test-coverage dimension** unless your CLAUDE.md demands one.

Where it falls down for our use case. No severity taxonomy at all. No thread resolution, no cross-run dedup, no re-run focus — its entire re-run strategy is step 1's blunt "abort if Claude already commented." **We confirmed this on a live repository** (§3.2.1, attempt 4): after it had reviewed a PR once, a subsequent run against a changed diff exited successfully having posted nothing. It reads `CLAUDE.md` but **not** `REVIEW.md`. Without `--comment` it posts nothing at all. And its required tools are not granted automatically — omit them and the review is silently discarded behind a green check.

⚠️ **Its own README is stale.** The README documents 0–100 confidence scoring with an 80 threshold and a git-blame history agent. Neither exists in the shipped command file — agent 4 is a second Opus bug hunter, and `git` is not even in its allowed tools. If anyone cites "confidence scoring" as a plugin feature, that claim comes from stale documentation.

Observed cost in the wild: Storybook's public workflow gates it behind a label with the comment *"Claude Code reviews are estimated to cost $15-25. use only when necessary and only internally."*

3.4 Managed "Code Review" (Anthropic SaaS)

What it is. *"Multiple agents analyze the diff and surrounding code in parallel on Anthropic infrastructure. Each agent looks for a different class of issue, then a verification step checks candidates against actual code behavior to filter out false positives."* Results are deduplicated, ranked by severity, and posted inline with a summary in the review body. Average completion: 20 minutes.

Four output surfaces, deliberately redundant: inline comments · review body summary (with an "Additional findings" section for lines that moved) · a check run with a full severity table · Files-changed annotations. Redundancy exists because *"Annotations and the severity table are written to the check run independently of inline review comments, so they remain available even if GitHub rejects an inline comment on a line that moved."*

Configuration via `REVIEW.md` (repo root only, injected as the highest-priority instruction block into every agent, taking precedence over the default guidance). Six documented levers: redefine severity · cap nit volume · skip rules by path/branch/category · repo-specific checks · raise the verification bar · **re-review convergence** ("after the first review, suppress new nits and post Important findings only") · summary shape. Warning from the docs: *"a long REVIEW.md dilutes the rules that matter most."*

Triggers changed in July 2026 — worth knowing if anyone is testing it: `@claude review` no longer subscribes the PR to push-triggered reviews. Use `@claude review always`.

Merge gating. *"The check run always completes with a neutral conclusion so it never blocks merging."* But it emits a machine-readable `<!-- bughunter-severity: {"normal":2,"nit":1,"pre_existing":0} -->` marker in the check-run output, so a team can build its own gate with `gh api` + `jq`.

Cost and limits. *"Each review averages $15-25 in cost, scaling with PR size, codebase complexity, and how many issues require verification"* — billed through usage credits, **not** covered by any subscription plan, and charged by Anthropic even for orgs otherwise on Bedrock or Google Cloud. Per-repo monthly spend caps exist; on breach, reviews are skipped with a PR comment until the next billing period. Runs are *"best-effort. A failed run never blocks your PR, but it also doesn't retry on its own"* — and GitHub's Re-run button does not retrigger it.

Blockers for us: research preview, **Team/Enterprise subscriptions only**, and **not available for organizations with Zero Data Retention enabled**. [unverified] The docs publish no supported-language list, repo-size ceiling, or maximum diff size.

3.5 Local `/code-review` and cloud `ultrareview`

Two developer-workstation tools, included because they change the "is there a free first-party option?" answer.

`/code-review` (local). Free on any plan, bundled into Claude Code. Reviews your branch's commits ahead of upstream plus uncommitted changes. `--fix` applies findings to the working tree; **`--comment` posts findings as inline PR comments**. Effort levels `low` → `max` trade false positives against coverage. Reads `CLAUDE.md` but **not** `REVIEW.md`. Runs as a background subagent (v2.1.218+) — note that `--fix` edits land outside session checkpoints, so `/rewind` will not undo them.

`/code-review ultra` (ultrareview). Research preview. A larger fleet of reviewer agents in a remote sandbox, where *"every reported finding is independently reproduced and verified."* Takes 5–10 minutes. Limits: **500 changed files and 8,000 changed lines**. Pricing: **3 free runs one-time for Pro/Max, none for Team/Enterprise**, then **$5–25 per review** in usage credits — and a run you stop early still consumes a free run.

Two disqualifiers for CI use: it **cannot post to a PR and cannot open a fix branch** (the docs redirect you to the managed service for that), and it **silently downgrades to a local review** on Bedrock, Google Cloud, Microsoft Foundry, or any ZDR organization — no error, just a quieter review.

3.6 GitHub Copilot code review

What it is. GA, and architecturally closer to our action than to a SaaS: it runs *"in an ephemeral development environment"* on **GitHub Actions runners**, using agentic `grep` / `rg` / `glob` / `view` tools to explore the repository. You can customize that environment with `.github/workflows/copilot-code-review.yml`, choose larger GitHub-hosted runners, or use self-hosted runners (Ubuntu x64 only, and self-hosted runners do not support the firewall feature).

Triggering. Manual (`Request` in the Reviewers sidebar, `gh pr edit --add-reviewer @copilot`, or the REST API against `copilot-pull-request-reviewer[bot]`), or automatic through **repository/organization/enterprise rulesets** — "Automatically request Copilot code review". Two sub-options, both **off by default**: "Review new pushes" and "Review draft pull requests". Without the first, *"Copilot will only review the pull request once."*

Configuration — genuinely rich. `.github/copilot-instructions.md`, path-specific `.github/instructions/**/*.instructions.md`, agent instruction files (`AGENTS.md`, and now `CLAUDE.md`, `GEMINI.md`, `REVIEW.md`), `.github/skills/` agent skills, and read-only MCP servers — the last two GA since 2026-07-29. Organization-wide custom instructions require Business or Enterprise. All of it is read from the **head branch**, so instruction changes can be tested in the same PR. No hard character cap is documented, only *"limit any single instruction file to a maximum of about 1,000 lines."*

Where it is genuinely better than us: the **metrics API**. Repo-level Copilot usage metrics went GA on 2026-07-17 and include `total_reviewed_by_copilot`, `total_copilot_suggestions`, `total_copilot_applied_suggestions`, and a per-comment-type breakdown — i.e. a real **suggestion acceptance rate**, which is the closest thing to an ROI number in this whole category. Our Backstage payload tracks findings and activity, but not whether a suggestion was accepted.

Where it fails our requirements, from GitHub's own docs:

- **Cannot gate.** *"Copilot always leaves a 'Comment' review, not an 'Approve' review or a 'Request changes' review. This means that Copilot's reviews do not count toward required approvals … and Copilot's reviews will not block merging changes."* Instructions to change this are explicitly listed as unsupported: *"Block a PR from merging unless all Copilot code review comments are addressed."*
- **Repeats resolved comments.** *"When re-reviewing a pull request, Copilot may repeat the same comments again, even if they have been dismissed with the 'Resolve conversation' button or downvoted with the thumbs down (👎) button."* There is no max-comments setting, no severity threshold, no nit suppression.
- **Ignores replies.** *"Any comments you add to Copilot's review comments will be visible to humans, but they won't be visible to Copilot, and Copilot won't reply."*
- **No severity taxonomy**, and formatting instructions are explicitly unsupported (*"Use bold text for critical issues"* is listed as a thing it will not do).
- **Permanent blind spots.** ~45 named files and 16 glob patterns are excluded and not configurable, including `tsconfig.json`, `*.d.ts`, `build.gradle`, `requirements.txt`, `**/vendor/**`, `**/generated/**`, `**/dist/**`.
- **Zero model control.** *"Model switching is not supported, as changing the model is likely to compromise reliability"*; and *"Copilot code review is an exception — the model is selected automatically and is not disclosed, so per-token costs may vary between reviews."* No BYOK.
- **No findings export API.** Aggregates via the metrics API; individual findings only through the standard PR review-comment REST endpoints.

Its own limitations page (the Copilot Agents application card) lists: *"Missed code quality problems"* · *"False positives: Copilot code review has a risk of hallucination"* · *"Inaccurate or insecure code suggestions"* · *"Potential biases … may be biased toward certain programming languages or coding styles."*

Data handling. Models are hosted by OpenAI/Azure and by AWS/Anthropic/GCP depending on selection, with zero-data-retention agreements for GA features. Data residency is available on GitHub Enterprise Cloud with data residency (US and EU) at a documented **+10% AI credit cost**. Content exclusions apply to code review, on Business/Enterprise only.

3.7 GitHub Code Quality — the adjacency that matters

Included because it is the likely response to "GitHub already gates quality for us."

GA since **2026-07-20**, **$10 per active committer per month plus usage**, on GitHub Enterprise Cloud and GitHub Team. Detection *"combines deterministic CodeQL rules for known anti-patterns with AI-powered analysis for issues that fall outside existing rule sets"* across C#, Go, Java, JavaScript, Python, Ruby and TypeScript. It gates merges through rulesets — "Require code quality results" with severity thresholds, plus coverage thresholds from Cobertura XML — and has a real findings REST API (public preview).

The load-bearing sentence for this whole evaluation: *"AI-powered findings never block your pull request on their own"* — the gate counts only rules-based CodeQL findings.

Read that together with §3.6: GitHub's own architecture concedes that an AI reviewer should not be the merge gate. Achieving full AI-review-plus-gating on GitHub's stack means stacking Copilot ($19–39/seat) **+** Code Quality ($10/committer) **+** optionally Code Security ($30/committer) — and the AI half still cannot gate.

---

4. Cost analysis

4.1 What we actually pay: the GLM Coding Pro plan (flat rate)

We do not pay per token. The org runs on z.ai's **GLM Coding Pro plan at roughly $80/month as billed** — a flat subscription with rolling usage windows. Within quota, **every review has zero marginal cost.**

z.ai moved from a prompt-based quota to a **credit** system; the doc's previous "~400 prompts per 5-hour window" figures no longer exist in z.ai's documentation and have been replaced:

Plan	5-hour credits	Weekly credits
Lite ($18/mo)	2,000	10,000
Pro (ours)	**12,000**	**60,000**
Max	28,000	140,000

Credits are computed from tokens: `(input × in-multiplier + cached input × cached-multiplier + output × out-multiplier) / 10,000`. For **GLM-5.2** the multipliers are **6.9 input / 1.7 cached input / 24 output**. Off-peak usage is charged at **50%**; peak is **Monday–Friday, 14:00–18:00 Singapore time (UTC+8)** — roughly 11:30–15:30 IST.

[unverified] Only the Lite price ($18) is quotable from z.ai's own documentation; the Pro (~$72–80) and Max ($160) prices come from secondary sources because z.ai's pricing page is client-rendered. Our billed figure of ~$80/month is the authoritative number for this brief.

4.2 What we actually run — 18 weeks of production telemetry

This is not a pilot. Measured from GitHub's Actions API across the six repositories running the action, **2026-04-12 → 2026-08-03**:

Metric	Value
Total runs recorded by GitHub	**2,768** across 6 repositories
Sustained weekly volume since mid-July	**~499 runs/week** (up ~8× from ~60/week in early July)
Implied monthly volume at that baseline	**~2,150 runs/month**
Busiest single day	196 runs
Successful reviews	1,682 (60.8% of all runs; 69.5% of runs that executed)
Cancelled mid-run (superseded pushes)	737 (26.6%)
Failed	192 (6.9%) · Skipped by design
Median duration	**9.4 min** (successful runs: median 11 min, mean 14 min, p95 37 min)
Total runner wall-clock	491 hours — **$0 billed**, self-hosted
Peak simultaneous runs	11; 79.2% of runs overlapped another

Two things this establishes for management: the action is **embedded in daily PR flow at production scale**, not being trialled; and the honest reliability picture is that **~13% of runs produce no review** (failed or skipped) with another ~27% cancelled by superseding pushes. §6 covers what to do about that.

4.3 What one review costs in credits

Using a reference token profile of **~245k input / ~38k output** for a full first review (see the sourcing note in §4.7):

```
(245,120 × 6.9 + 38,440 × 24) / 10,000  =  ~261 credits per review
                               off-peak  =  ~131 credits per review
```

Capacity on the Pro plan:

Window	Peak rate	Off-peak rate
Per 5-hour window (12,000 credits)	~46 reviews	~92 reviews
Per week (60,000 credits)	~230 reviews	~460 reviews
Per month (approx.)	**~1,000 reviews**	**~2,000 reviews**

Reconciling that ceiling with the ~2,150 runs/month we actually do (§4.2). Runs are not all billable reviews, and the reference profile is a *full first review* — the most expensive kind:

Of ~2,150 runs/month	Share	AI quota consumed
Skipped by design (oversized PR, no agents)	~5.6%	none — the skip happens before any agent call
Failed	~6.9%	little — median failure ends in ~1 minute
Cancelled by a superseding push	~26.6%	partial — median 4.2 min, killed mid-review
Completed reviews	**~69.5% ≈ 1,500/month**	full, and cheaper on re-runs (description and diagram calls skipped, inline scope narrowed)

So ~1,500 billable reviews/month sits inside the 1,000–2,000 band above, which is exactly what we observe: **the Pro plan has carried this volume in production for four months.** The band is wide because it depends on the peak/off-peak mix — our peak window (11:30–15:30 IST) is mid-workday, so a meaningful share of reviews pay the 2× rate.

Treat 1,000–2,000/month as a planning envelope, not a hard ceiling. If volume grows past it, the Max plan roughly doubles the weekly allowance for $160/month — still lunch money at org scale.

Amortized cost per review on ~$80/month: ~$0.05 at our current ~1,500 completed reviews/month · ~$0.08 at 1,000 · ~$0.16 at 500.

Compute is free to us as well. Our action runs on **SourceFuse self-hosted runners**, so it consumes **no billable GitHub Actions minutes** — the AI subscription is the entire cost. This is a real line-item difference from Copilot code review, which has run on GitHub Actions runners and billed minutes to the repository on private repos since 2026-06-01, on top of its seat and AI-credit charges.

Quota exhaustion degrades gracefully by design. z.ai returns 429s until the window resets; our patient rate-limit budget (10s escalating ×1.5 to a 120s ceiling, up to 5h per AI call) rides out the reset. A throttled review is late, never lost.

4.4 API-equivalent benchmark — *not what we pay*

This table exists solely to compare like with like against per-review-priced competitors. **We are on the flat Pro plan; none of these API prices is a bill we receive.**

Option	Cost per review	Basis
Ours on the GLM Coding Pro plan (actual)	**$0 marginal** (~$0.08–0.40 amortized)	§4.2
Ours on Claude Haiku 4.5 API	~$0.44	$1 / $5 per MTok
Ours on GLM-5.2 pay-per-use API	~$0.51	**$1.4 / $4.4** per MTok
Ours on Claude Sonnet 5 API	~~$0.87 → **~~$1.31 from 2026-09-01**	$2/$10 introductory, then $3/$15
Ours on Claude Opus 5 API — the frontier-quality option, one line of YAML	**~$2.19** (~$2.85 allowing for the newer tokenizer)	$5 / $25 per MTok
ultrareview	$5–25	3 free runs (Pro/Max), then usage credits
Managed Code Review	**$15–25**	*"Each review averages $15-25 in cost, scaling with PR size"*
Copilot code review	**not disclosed**	see §4.4

Two adjustments worth knowing:

- The previous version of this document priced GLM-5.2 at $0.60/$2.20. **That is GLM-4.7's price.** GLM-5.2 is $1.4/$4.4, so the API-equivalent figure roughly doubles — it does not change the conclusion, but the old number was wrong.
- Claude models from 4.7 onward use a newer tokenizer that *"produces approximately 30% more tokens for the same text."* Any Opus 5 / Sonnet 5 estimate built from older token counts is understated by roughly that much; the table shows both.

4.5 Copilot's cost is not forecastable — and that is a documented fact

Copilot code review has **three** cost components and GitHub publishes a number for none of them at the per-review level:

1. **Seat**: $10 Pro · $39 Pro+ · $100 Max · **$19 Business** · **$39 Enterprise** per user per month. Annual billing for Business/Enterprise is not offered on the current usage-based model (legacy annual plans are grandfathered only). [unverified] Volume discounts would be a sales negotiation.
2. **AI credits** at $0.01 each, pooled at the billing entity (1,900/user Business, 3,900/user Enterprise) and **forfeited monthly if unused**. Promotional allowances (3,000 and 7,000) **end 2026-09-01**. The per-review credit consumption is undisclosed because *"the model is selected automatically and is not disclosed."*
3. **GitHub Actions minutes**, charged to the repository on private repos, since 2026-06-01. Minutes per review: not published. Medium effort *"uses more AI credits and GitHub Actions minutes than Low."* Our action carries no equivalent charge — it runs on our self-hosted runners (§4.3).

The only public per-review anchor is the **retired** premium-request model: code review consumed *"13 premium requests"* at $0.04 each = **$0.52 per review** — and that model was replaced on 2026-06-01, so it applies only to grandfathered subscribers.

The fair framing for management: if the org already pays for Copilot seats for code completion, review commentary is close to free at the margin — but it is *unmeasurable* commentary that cannot gate a merge, cannot be exported per finding, and repeats comments developers have already resolved.

4.6 What it costs at our volume

Anchored on the volume we actually run (§4.2): **~1,500 completed reviews per month** across 6 repositories.

Completed reviews / month	**Ours (GLM Pro plan)**	Ours @ Opus 5 API	Managed Code Review	Copilot code review
500	**~$80 flat**	~$1,090–1,420	$7,500–12,500	seats (likely sunk) + unforecastable usage + Actions minutes
1,000	**~$80 flat**	~$2,190–2,850	$15,000–25,000	as above
1,500 — our current volume	**~$80–160** (Pro, Max for headroom)	~$3,280–4,270	**$22,500–37,500/mo = $270k–450k/yr**	as above

At our real volume, the managed service would cost roughly **$270,000–450,000 per year** for the six repositories currently running the action. Our total AI spend for the same work is **~$960–1,920 per year.** And because the managed service's "after every push" mode bills *every push*, its realistic figure is higher still — our telemetry shows 27% of runs are superseded by another push, each of which would be separately billed.

The cost ladder: GLM plan (≈$0 marginal) → GLM API ($0.51) → Sonnet 5 ($0.87–1.31) → **Opus 5 ($2.19–2.85)** → ultrareview ($5–25) → managed service ($15–25). Even the premium play — our pipeline with Opus 5 as the reviewer — is **5–11× cheaper per review than the managed service**, while keeping every workflow feature. Because the model is a config value, that quality/cost trade is a one-line change per repo, not a platform migration.

4.7 Sourcing notes and dated caveats

- Our per-review token profile (~245,120 in / ~38,440 out, 7 AI calls) is the **illustrative payload in `docs/backstage-integration.md`**, not captured production telemetry. It is internally consistent with our cost estimator, but it should be replaced with a real run before being quoted externally. Our per-run token and cost figures are client-side estimates, never billing data — every summary comment says so.
- A default combined-mode first run makes roughly **4–5 AI calls** (pre-flight, one agent, description, diagrams); separate/strict mode reaches ~12. Re-runs are cheaper — the description and diagram calls are skipped entirely.
- **Volume, latency and outcome figures in §4.2 are real**, derived from GitHub's Actions API across all 2,768 runs of the workflow in the six repositories, 2026-04-12 → 2026-08-03. Durations are wall-clock (`run_started_at` → `updated_at`), which includes queue and idle time — a consistent relative measure, not billable minutes. Runs under 1s or over 2h are excluded from duration statistics.
- Our per-review **token** figures remain the weak link: no fleet-wide token telemetry is aggregated yet, so the credit math in §4.3 rests on a single reference profile. The action already emits `input_tokens` / `output_tokens` per run; aggregating them through the Backstage payload would replace the estimate with measurement.
- **Dated items to re-verify before quoting:** Claude Sonnet 5 rises from $2/$10 to $3/$15 on **2026-09-01**; Copilot Business/Enterprise promotional AI credits drop on **2026-09-01**; z.ai's off-peak discount and multipliers are subject to change; Anthropic's managed Code Review is a research preview and its pricing may move.
- [unverified] Claude Max subscription pricing ($100/$200) and whether a `claude_code_oauth_token` used in CI draws on the same quota as a developer's interactive Claude Code session. The latter is a strong inference — the OAuth token authenticates the same subscription identity — but it is not stated in any primary source.

---

5. Scoring

0–5 per dimension, weighted to 100. Weights are unchanged from the previous version of this document so the two can be diffed. Disagree with a weight? The raw scores let you re-weight.

Dimension (weight)	Ours	OSS	Plugin	Managed	Copilot	Notes
Review engine features (12)	4	2	3	5	3	Managed: multi-agent + verification. Ours: 8 dimensions, dedup, profiles. Plugin: multi-agent but deliberately narrow. Copilot: no taxonomy, no profiles
False-positive control (8)	3	2	4	5	2	Plugin and Managed both verify each finding; ours does not, and our prompts bias toward flagging. Copilot's own docs cite hallucination risk
PR workflow automation (12)	5	2	1	4	2	Only ours: re-run focus, reply verification, reopen-on-regression, description + diagrams. Plugin aborts rather than converges
Codebase context depth (8)	4	4	4	5	4	Theirs read broadly and uncapped; ours is compiler-exact and cost-bounded; Copilot is agentic but has permanent file blind spots
Metrics & governance (10)	5	2	1	3	3	Only ours: per-finding tracking + 23 outputs + cost in-PR. Copilot has the better *aggregate* metric (acceptance rate) but no findings export
Cost per review (12)	5	3	3	1	3	~$0 marginal on the existing plan vs $15–25 managed vs Copilot's undisclosed usage
Provider & model flexibility (8)	5	4	4	1	0	Copilot: model undisclosed, unselectable, no BYOK
Org fit — JIRA, Backstage, z.ai, Angular/LB4, gating (10)	5	1	1	1	2	Copilot scores 2 for read-only MCP context; none of the others covers any of our integrations
Noise control / developer experience (6)	5	2	3	4	1	Copilot documents that it repeats resolved comments and offers no volume control
Maintenance & support (6)	2	5	5	5	5	Honest: we maintain ours
Maturity / stability (4)	4	4	3	2	5	Ours in production here; OSS is GA; managed is research preview; Copilot is GA at enormous scale
Security & data residency (4)	5	4	4	2	4	Ours: our CI + our endpoint. Managed: Anthropic infra, no ZDR orgs. Copilot: US/EU residency at +10%
Weighted total (100)	**88**	**54**	**55**	**64**	**53**	

---

6. Risks and counterarguments

1. **"Anthropic and GitHub maintain theirs; who maintains ours?"** True, and the single largest real cost of this decision. Mitigations: the codebase is deliberately boring to maintain — `action.yml` is generated from one schema and drift-checked at build time, all tunables live in one constants file, the summary-comment output is snapshot-locked, and 251 tests run on `npm run ci`. The surfaces we depend on (GitHub REST/GraphQL and one streaming chat API) are stable. **Gap to close:** this repository has no `.github/` workflows, so the drift check and test suite run only on a developer's `npm run build`, not in CI. That should be fixed.
2. **"Isn't Claude a better reviewer than GLM?"** On raw model quality, likely yes — and the answer is **Opus 5 through our own pipeline**, not a different platform. Pointing `anthropic_base_url` at Anthropic turns our action into an Opus 5 reviewer for ~$2.19–2.85 per review: frontier-model quality, still 5–11× under the managed service, with every workflow feature intact. At our scale, review *usefulness* is dominated by workflow — noise control, thread lifecycle, tracking — which is where ours leads regardless of model.
3. **"We already pay for Copilot; isn't its review free?"** Nearly free at the margin, yes — but it cannot gate a merge, has no severity taxonomy, ignores developer replies, re-posts comments developers already resolved, permanently skips files like `tsconfig.json` and `*.d.ts`, and gives you no per-finding export for tracking. It is worth running **alongside** ours on repos where the seats are already paid; it is not a replacement for the gating and tracking layer.
4. **"The managed service will improve fast."** Agreed — it is a research preview and worth re-evaluating in ~6 months. Its *pricing model* (per review, per push) is the structural problem, not its quality. Note also that it is unavailable to ZDR organizations, which may be a hard blocker for some client engagements.
5. **"Aren't we duplicating their roadmap?"** Partially, in both directions. Their best idea we lack — a verification pass to kill false positives — is a bounded addition to our pipeline (§7.3). Our operational features (JIRA, Backstage, gating, reply verification, provider freedom) are not on anyone's public roadmap and arguably never will be, since several of them conflict with a managed business model.
6. **Our own reliability numbers are not flattering, and management should see them.** Across 2,768 production runs: **6.9% failed** (192 runs, and weekly failures ran 29 → 43 → 118 across three flat-volume weeks — growing faster than usage), and **56 runs hung past two hours** before being killed, concentrated in a single three-day window. Both are addressable (§7.6).

   The **26.6% cancellation rate** (737 runs, 111 runner-hours) needs a correction that an earlier draft of this document got wrong. Those are not a missing-configuration problem: a `concurrency` group with `cancel-in-progress: true` has been in the workflow since **2026-04-12**, the first day of the measurement window. The cancellations *are* that mechanism working — a developer pushes again while a review is in flight, and the superseded run is killed. GitHub cannot avoid starting the first run, so this cost is inherent to a push-heavy workflow rather than a defect. The lever that would actually reduce it is behavioural (fewer rapid successive pushes) or a short debounce, not a config flag.

   It is worth noting that the competitors' equivalent failure rates are simply not observable: none of them publishes one, and only ours emits the telemetry that makes this table possible.
7. **What we genuinely do not have.** A false-positive verification pass. A suggestion-acceptance-rate metric. 👍/👎 feedback learning. A hosted analytics UI (we have a Backstage payload; someone must build the dashboard). "Fix this" deep links. And the entire interactive dev-automation category — writing code, creating PRs, fixing bugs on request, `@mention` Q&A, MCP servers, scheduled automation. All of that is out of scope for a reviewer by design, and it is why `claude-code-action` is a **complement** to ours rather than a competitor.

---

7. Recommendation

1. **Keep our AI PR Review Action as the org-wide default reviewer.** It is the only option that satisfies our integrations (JIRA, Backstage, Angular/LoopBack4), runs at **zero marginal cost on the existing ~$80/month GLM Coding Pro plan**, controls review noise across re-runs, and can gate a merge at all.
2. **For quality-critical repos, upgrade the model, not the platform.** Point the same action at **Claude Opus 5** via `anthropic_base_url` + `anthropic_model` for ~$2.19–2.85 per review — frontier-model reviews with all our workflow features, one line of YAML per repo.
3. **Adopt the competitors' best idea: add a verification pass.** Three of the five Anthropic offerings now verify each candidate finding with an independent agent before posting. A bounded "try to refute this finding" step before we post inline comments closes our one meaningful quality gap at a marginal token cost, and directly attacks the false-positive complaint that drives developer distrust.
4. **Adopt Copilot's best idea: measure acceptance.** Copilot's metrics API exposes a suggestion acceptance rate; our Backstage payload does not. Adding an "was this suggestion committed?" signal to the Backstage schema would give us the ROI number this category otherwise lacks.
5. **Run Copilot code review alongside, where seats are already paid.** It costs nothing extra at the margin on repos whose developers hold Copilot seats, and a second opinion has value. Configure it via ruleset with "Review new pushes" **off** to limit the documented comment repetition. Do not treat it as a gate; it cannot be one.
6. **Apply the two reliability fixes the telemetry actually supports:**
   - an explicit **`timeout-minutes: 45–60`** on the review job — the 95th percentile of successful runs is 37 minutes, so this truncates the hung-run tail (56 runs, 247 wall-clock hours) without touching healthy reviews. This is genuinely missing today;
   - **triage the failure mode** driving the 29 → 43 → 118 weekly trend before volume grows further.

   Explicitly *not* recommended: adding a `concurrency` group. It is already there and has been since day one (§6.6).
7. **Aggregate token telemetry into Backstage.** The action already emits `input_tokens` / `output_tokens` per run; nothing consumes them in aggregate. Doing so replaces the single reference profile behind our credit math (§4.3) with measurement, and gives finance a real cost-per-review number.
8. **Fix the CI gap in our own repo** (no `.github/` workflows). The maintenance argument in §6.1 depends on the drift check and test suite actually running on every PR.
9. **Re-evaluate in 6 months.** The managed service will mature and Copilot ships monthly. Our per-review cost advantage and integration moat are unlikely to change, but the verification-quality gap may narrow from both directions.

Appendix A — our configuration surface

	Ours	claude-code-action	Copilot code review
Inputs	**49**, all review-specific	**39** (plus ~10 legacy inputs migrated to `claude_args`); review config lives in a free-form `prompt`	none — repo settings, rulesets, and instruction files
Config style	Typed, validated, generated `action.yml` (drift-checked at build)	Prompt engineering per workflow + `settings` JSON	Markdown instruction files + ruleset toggles
Outputs	**23** named review outputs	5 (`execution_file`, `branch_name`, `github_token`, `structured_output`, `session_id`)	none

Our 49 inputs by group: required ×2 (`github_token`, `anthropic_auth_token`) · provider ×8 (`ai_provider`, `anthropic_base_url`, `anthropic_model`, `model_pricing`, `max_tokens`, `thinking_budget`, `context_window`, `temperature`) · review mode ×2 · individual review toggles ×8 · framework ×1 · GitHub ×1 (`pr_number`) · JIRA ×4 · behavior flags ×12 · prompt customization ×4 · comment ×2 · diagrams ×1 · advanced ×4.

Our 23 outputs: `review_status`, `skip_reason`, `review_comment_id`, `review_comment_url`, `total_findings`, `critical_count`, `high_count`, `medium_count`, `low_count`, `nit_count`, `review_passed`, `agents_run`, `agents_failed`, `duration_seconds`, `backstage_reported`, `replies_posted`, `threads_resolved_from_replies`, `threads_reopened`, `bot_comments_hidden`, `ai_calls`, `input_tokens`, `output_tokens`, `estimated_cost_usd`.

Appendix B — where our figures come from

Claim	Source of truth
49 inputs / 23 outputs	`src/config/schema.ts` → generated `action.yml`
5 severities, 8 finding categories	`src/config/taxonomy.ts`
Profiles 8 / 6 / 2 agents	`src/config/profiles.ts`
Dedup thresholds (±2 lines, Levenshtein ≥ 0.65, Jaccard ≥ 0.5, skip AI pass at ≤ 3 findings)	`src/config/limits.ts`
Agentic tool caps (2 rounds combined / 1 separate, 6 per review, 12 per run)	`src/config/limits.ts`, `src/context/local/context-tools.ts`
Rate-limit budget (10s ×1.5 → 120s ceiling, 5h per AI call)	`src/config/limits.ts`, `src/providers/base-provider.ts`
Inline comments use `event: 'COMMENT'`	`src/github/inline-reviewer.ts`
Re-run inline scope (critical/high + documentation)	`src/config/taxonomy.ts`
Backstage payload fields	`src/results/backstage-reporter.ts`, `docs/backstage-integration.md`
Reference token profile (245k/38k)	`docs/backstage-integration.md` (**illustrative payload, not telemetry**)
Production volume, latency, outcome mix (§4.2)	GitHub Actions REST API, all 2,768 runs of `ai-code-review.yml` across 6 repos, 2026-04-12 → 2026-08-03
251 tests	`npm test`

Appendix C — external sources (all fetched 2026-08-03)

Anthropic

- https://code.claude.com/docs/en/github-actions — action overview, plugin workflow, `claude_args` (`--max-turns` default 10), Bedrock/Vertex/Foundry/OIDC auth, Agent SDK
- https://code.claude.com/docs/en/code-review — managed service: $15–25/review, 20-minute average, research preview, Team/Enterprise, no ZDR, 🔴/🟡/🟣 taxonomy, `REVIEW.md` levers, `bughunter-severity` check-run JSON, spend caps, analytics dashboard, July 2026 `@claude review always` change
- https://code.claude.com/docs/en/ultrareview — 3 free runs, $5–25, 5–10 minutes, 500 files / 8,000 lines, cannot post to a PR, silent downgrade on Bedrock/Vertex/Foundry/ZDR
- https://code.claude.com/docs/en/commands — local `/code-review` flags, `--comment`, `/simplify` history
- https://github.com/anthropics/claude-code-action/discussions/673 — "Anyone tried using Claude Code GitHub Actions with Z.AI (GLM models)?" — the community-verified working configuration cited in §3.2.1
- https://docs.z.ai/devpack/tool/claude — z.ai's own Claude Code setup: `ANTHROPIC_AUTH_TOKEN` + `ANTHROPIC_BASE_URL: https://api.z.ai/api/anthropic` + `API_TIMEOUT_MS`, i.e. two of the three variables the action drops
- https://github.com/anthropics/claude-code-action — `action.yml` (39 inputs, 5 outputs, the ~75-variable env allow-list — `ANTHROPIC_BASE_URL` and `ANTHROPIC_CUSTOM_HEADERS` present, `ANTHROPIC_AUTH_TOKEN` and `API_TIMEOUT_MS` absent), `docs/solutions.md` (8 recipes), `docs/capabilities-and-limitations.md`, `docs/usage.md`, `docs/setup.md` (workload identity federation), `src/mcp/github-inline-comment-server.ts`, `src/entrypoints/post-buffered-inline-comments.ts` (`claude-haiku-4-5` classifier)
- https://github.com/anthropics/claude-code/blob/main/plugins/code-review/commands/code-review.md — the real 9-step pipeline; its `README.md` is stale
- https://platform.claude.com/docs/en/about-claude/pricing — Opus 5 $5/$25, Fable 5 $10/$50, Sonnet 5 $2/$10 → $3/$15 on 2026-09-01, Haiku 4.5 $1/$5, ~30% tokenizer increase on 4.7+, batch −50%, caching, fast mode

z.ai

- https://docs.z.ai/devpack/overview — credit allowances (2,000/12,000/28,000 per 5h; 10,000/60,000/140,000 weekly), credit formula, GLM-5.2 multipliers 6.9/1.7/24, 50% off-peak, peak Mon–Fri 14:00–18:00 UTC+8
- https://docs.z.ai/guides/overview/pricing — GLM-5.2 $1.4 input / $0.26 cached / $4.4 output

GitHub

- https://docs.github.com/en/copilot/concepts/agents/code-review — ephemeral Actions environment, agentic context, effort levels, unlicensed-user reviews, MCP, budget cutoff
- https://docs.github.com/en/copilot/how-tos/use-copilot-agents/request-a-code-review/use-code-review — "Comment" review only, repeated comments on re-review, replies invisible to Copilot, `copilot-pull-request-reviewer[bot]`, "usually less than 30 seconds"
- https://docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/configure-automatic-review — rulesets, "Review new pushes", "Review draft pull requests"
- https://docs.github.com/en/copilot/reference/review-excluded-files — the permanent exclusion list
- https://docs.github.com/en/copilot/tutorials/customize-code-review — supported and unsupported instruction types
- https://docs.github.com/en/copilot/responsible-use/agents — missed problems, false positives, insecure suggestions, bias
- https://docs.github.com/en/copilot/reference/copilot-billing/models-and-pricing — model not disclosed; AI credits + Actions minutes
- https://docs.github.com/en/copilot/get-started/plans — $10 / $39 / $100 / $19 / $39 seat pricing
- https://docs.github.com/en/copilot/concepts/billing/usage-based-billing-for-organizations-and-enterprises — $0.01/credit, pooling, monthly forfeiture, promo end 2026-09-01
- https://docs.github.com/en/copilot/reference/copilot-usage-metrics/copilot-usage-metrics — `total_reviewed_by_copilot`, `total_copilot_applied_suggestions`
- https://docs.github.com/en/code-security/concepts/code-quality/code-quality and https://docs.github.com/en/code-security/tutorials/improve-code-quality/catch-issues-before-merge — GA, $10/committer, ruleset gating, **"AI-powered findings never block your pull request on their own"**
- https://github.blog/changelog/ — 2026-03-05 (agentic architecture GA), 2026-04-27 (Actions minutes from 2026-06-01), 2026-06-25 (~20% cost reduction), 2026-07-17 (customization; repo-level metrics GA), 2026-07-20 (Code Quality GA), 2026-07-29 (skills + MCP GA)

Prepared 2026-08-05 by the Platform Engineering team. Competitor pricing and preview status should be re-verified before external quotation; see the dated caveats in §4.6.